Week 9: Adversarial Search and Game Playing Agents

Learning Outcomes

By the end of this lab, students will be able to:

- Formally model a **two-player deterministic game**
- Implement **Minimax search**
- Optimize search using **Alpha-Beta pruning**

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Formal Game Representation (Foundational Task)

Problem Statement

Design and implement a **formal computational model** of a two-player, turn-based, deterministic, zero-sum game.

Requirements

Your implementation must explicitly define the following components as separate functions/modules:

1. getInitialState()
    - Returns the starting configuration of the game.
2. getPlayer(state)
    - Returns which player is to move (MAX or MIN).
3. getActions(state)
    - Returns all legal actions available in the current state.
4. getResult(state, action)
    - Returns the new state after applying the given action.
5. isTerminal(state)
    - Returns true if the game has ended; otherwise false.

6. getUtility(state)
    - Returns:
        - +1 if MAX wins
        - -1 if MIN wins
        - 0 for draw

Constraints

- Use a **simple game** such as:
    - Tic-Tac-Toe (recommended), OR
    - Any custom grid-based deterministic game

Expected Output

- Demonstration of:
    - Valid moves
    - State transitions
    - Terminal detection

Task 2: Game Tree Generation

Problem Statement

Construct the **complete game tree** starting from the initial state up to terminal states.

Requirements

- Use recursion to:
    - Expand all possible states
    - Represent each node as a game state
- Store:
    - Parent-child relationships
    - Depth of each node
- Print:
    - Total number of nodes generated
    - Maximum depth of the tree

Task 3: Implementation of Minimax Algorithm

Problem Statement

Implement the **Minimax algorithm** to compute the optimal move assuming both players play optimally.

Requirements

You must implement:

- minimaxDecision(state)
- maxValue(state)
- minValue(state)

Functional Expectations

- Recursively traverse the game tree
- Propagate utility values from terminal states to root
- Select the action that maximizes the utility for MAX

Output Requirements

For a given state, display:

- Best action selected
- Minimax value
- Number of nodes explored

Task 4: Depth-Limited Minimax with Predefined Heuristic Evaluation

Problem Statement

Extend your previously implemented Minimax algorithm (Task 3) to support **depth-limited search** by incorporating a **predefined heuristic evaluation function**. The objective is to enable decision-making in scenarios where full game tree expansion is computationally infeasible.

Implementation Instructions

1. Reuse Existing Implementation

You are strictly required to **reuse your Minimax implementation from Task 3**. Do not implement a new algorithm from scratch.

2. Modify Function Signatures

Update your recursive functions to include a depth parameter:

- maxValue(state, depth)
- minValue(state, depth)

3. Stopping Criteria

Terminate recursion when either of the following conditions is satisfied:

- The current state is terminal (isTerminal(state) == true), OR
- The specified depth limit is reached (depth == DEPTH_LIMIT)

In case the depth limit is reached before reaching a terminal state, you must invoke the **evaluation function** instead of the utility function.

4. Mandatory Heuristic Evaluation Function

You must implement the following **fixed evaluation function for Tic-Tac-Toe**:

Evaluation Rules

Evaluate all rows, columns, and diagonals based on the following scoring scheme:

Pattern Configuration	**Score Contribution**
Three marks of MAX (X X X)	+100
Two marks of MAX and one empty cell	+10
One mark of MAX and two empty cells	+1
Three marks of MIN (O O O)	-100
Two marks of MIN and one empty cell	-10
One mark of MIN and two empty cells	-1

5. Evaluation Procedure

- Examine all possible winning lines (3 rows, 3 columns, 2 diagonals).
- Compute the score for each line independently.
- Return the **sum of all line scores** as the heuristic value of the state.

6. Output Requirements

For a given game state, your program must display:

- The selected optimal move
- The computed evaluation value
- The depth at which evaluation was performed

7. Experimental Analysis

Execute your program using the following depth limits:

- DEPTH_LIMIT = 1
- DEPTH_LIMIT = 2
- DEPTH_LIMIT = 3

Task 5: Optimization using Alpha-Beta Pruning

Problem Statement

Enhance your existing Minimax implementation (Task 3 / Task 4) by incorporating **Alpha-Beta pruning** to eliminate redundant branches in the game tree, thereby improving computational efficiency without affecting the optimal decision.

Important Instruction

You must **modify your existing Minimax implementation**. Do **NOT** implement a separate or independent algorithm.

Implementation Instructions

1. Extend Function Signatures

Modify your functions as follows:

- maxValue(state, alpha, beta)
- minValue(state, alpha, beta)

Initialize at the root:

- alpha = -∞
- beta = +∞

2. MAX Node Logic

Within maxValue(state, alpha, beta):

- Initialize:

    - v = -∞
- For each successor state:
    - Compute:
        - v = max(v, minValue(child, alpha, beta))
- Apply pruning condition:
    - If v ≥ beta, terminate further exploration of this branch
- Update:
    - alpha = max(alpha, v)

3. MIN Node Logic

Within minValue(state, alpha, beta):

- Initialize:
    - v = +∞
- For each successor state:
    - Compute:
        - v = min(v, maxValue(child, alpha, beta))
- Apply pruning condition:
    - If v ≤ alpha, terminate further exploration of this branch
- Update:
    - beta = min(beta, v)

4. Mandatory

You must maintain the following counters:

- Total number of nodes visited
- Total number of nodes pruned

These values must be displayed at the end of execution.

5. Comparative Experiment

Execute both algorithms on the same initial game state:

1. Standard Minimax (from Task 3)

2. Alpha-Beta Pruning version

6. Output Format

Present results in the following tabular format:

Algorithm	**Nodes Visited**	**Nodes Pruned**	**Optimal Move**	**Utility Value**
Minimax		0		
Alpha-Beta				